Notes on pySAS006 Data processing

July 2026

Simon Bélanger

Summary:

I developed a pipeline to automate the multi-level processing (RAW to L2) of SeaBird/pySAS data from our pySAS006 using NASA's HyperCP framework.

The pipeline bypasses the manual GUI configurations, filters datasets dynamically by date, time, and implements a pre-processing step to correct sensor data anomalies (heading dropouts and tilt biases) natively in Python before quality control execution. It does not yet add the header to the RAW file, which is still being done using R.

This pipeline replaces the manual GUI steps and handles data quality corrections automatically. Below is a detailed breakdown of how the scripts work and how you should use them.

1. Architecture: What the Scripts Do

The pipeline is split into two modular scripts to keep the automation clean and easy to maintain.

- **correct_L1A_data.py (Pre-processing & Calibration Patching; call by the next script)**
  - **Time-Aware Circular Interpolation**: It automatically detects dropouts, missing blocks, or instrument errors (NaN or -9999 fill values) in the HEADING dataset. It projects the angles into sine/cosine components to compute a gap-free circular interpolation without phase jumps (0 to 360° boundary fixes).
  - **Physical Bias Correction**: It corrects a biais in the IMU noticed by Matthieu, applying a vertical offset of -5 on the ROLL dataset to correct the physical misalignment of the vessel's sensor tower.
  - **Automated diagnostic plots**: It generates three-color verification plots (darkgray for original data, forestgreen for valid data, and crimson for interpolated values) and saves them in a structured /plots/ directory for visual inspection of the dataset's integrity.
  -
- **run_pySAS006_processing.py (Main Pipeline Automation)**
  - **Dynamic Argument Parsing**: Eliminates hardcoded variables by accepting dates and targeted data levels directly via the terminal.
  - **Configuration Patching**: It parses, edits, and overwrites the master JSON configuration files (.cfg) and SeaBASS header profiles (.hdr) on the fly based on your selected L2 optical model version.
  - **Interception Loop**: If processing the Level-1A Quality Control (L1AQC) stage, it automatically intercepts the execution path, targets the original L1A folder, triggers the script above, and funnels the updated outputs into an L1A_interpolated directory before starting HyperCP.
  - **Parallel Execution**: Utilizes multithreading routines (multiprocessing) to process a day's worth of files simultaneously.

2. Directory Structure After Execution

The script maintains a strict, standardized data tree underneath your parent directory:

```
pySAS/
├── RAW/                # Raw sensor binary streams (.raw)
├── L1A/                # Standard Level-1A files (.hdf)
├── L1A_corrected/      # Corrected data (Fixed Heading & Roll)
│   └── plots/          # Visual QC plots (.png)
├── L1AQC/              # Quality-controlled files passed by HyperCP
├── L1B/ / L1BQC/       # Intermediate levels
└── [L2_VERSION]/       # Final L2 products (e.g., M99SimSpec, Z17NIR)
    ├── config_run_[...].cfg  # Archived metadata for processing traceability
    └── header_run_[...].hdr  # Saved headers used for this run
```

3. How to run the code

Always make sure your virtual environment is active before running the script:

```
conda activate hypercp
```

Navigate to the processing scripts directory (in my case: `HyperCP/MyScripts/`).

The pipeline is built to process datasets flexibly: you can run an entire day at once or isolate a specific single cast by appending the optional `--time` flag.

I strongly recommend to run the code Step by step and pay attention to the processing verbose. Sometime it is the only way to know why some of your data do not reach the L2… Often, you may need to modify the processing parameters depending on your environment!!

To process the files from July 1st, 2026, execute the following sequential commands:

Step 1: RAW to L1A

- `python run_pySAS006_processing.py --date "20260701" --level "L1A"`
  - *Optional time-targeting:* `python run_pySAS006_processing.py --date "20260701" --time "162336" --level "L1A"`
  - *Note:* Very quick execution.

Step 2: L1A to L1AQC (Pre-processing Calibration & QC)

- `python run_pySAS006_processing.py --date "20260701" --level "L1AQC"`
  - *Note:* Very quick execution. This critical step calls `clean_L1A_data.py` to fix the progressive pySAS tower heading dropouts via a time-aware circular interpolation, subtracts the static -5° IMU `ROLL` alignment offset, and pipes the corrected files into `L1A_corrected/` right before HyperCP screen filtering.

Step 3: L1AQC to L1B

- `python run_pySAS006_processing.py --date "20260701" --level "L1B"`

- *Note:* This step is time-consuming as it handles multi-instrument temporal binning.

Step 4: L1B to L1BQC

- `python run_pySAS006_processing.py --date "20260701" --level "L1BQC"`
  - *Note :* This step could eliminates significant amount of data and it is important to carefully examine the processing VERBOSE to identify any issue with the processing parameters. EXAMPLE: default setting eliminate LT spectra where LT(UV) < LT (NIR) (`"bL1bqcLtUVNIR": 1`)… but this will remove turbid waters!!!

Step 5: Level-2 Reflectances (R_{rs})

You can process a single target method by declaring its name (e.g., `--version "M99SimSpec"`). However, to run all combinations sequentially in a fully automated super-batch, invoke the **`ALL`** keyword:

- `python run_pySAS006_processing.py --date "20260701" --level "L2" --version "ALL"`
  - *Note:* The `ALL` loop automatically generates independent, unconflicted directories for each successful algorithmic variant, logs execution steps, and archives the respective `.cfg` and `.hdr` configuration records alongside the final HDF5 products for total scientific traceability.

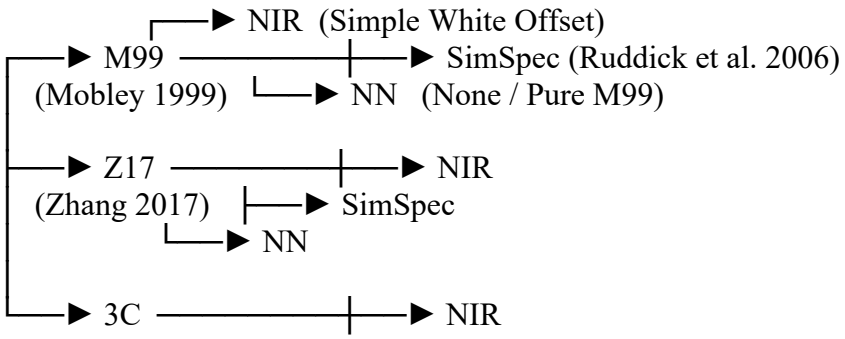
WARNING: This step can be time consuming if run with ALL in the current version is running 9 versions:

```
liste_versions = ["M99SimSpec", "M99NIR", "M99NN", "Z17SimSpec", "Z17NIR", "Z17NN", "3CSimSpec",
                  "3CNIR", "3CNN"]
```

  - This line (around 321) can be edited to select the most appropriate methods for a given environment.

As you can see for L2, you have to choose the RhoSky configuration
The Level-2 processing converts the quality-controlled water-leaving radiances by removing the surface sky reflection (glint). It is defined by a combinations of two distinct physical choices: **The rho_sky Surface Reflectance Method** (3 options) and **The Residual NIR/White Offset Correction** (3 options).

L2 VERSION MATRIX (3 x 3 = 9 Combinations)



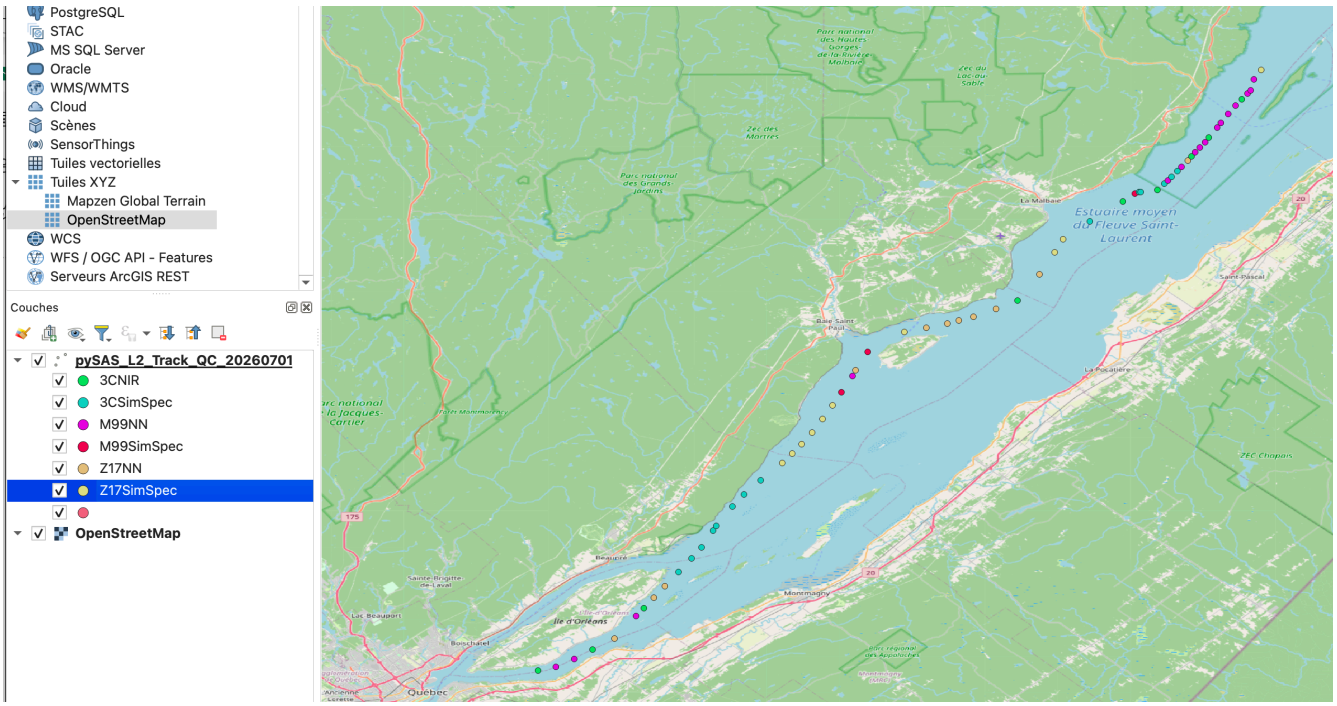
(Pitarch 2020) ├──► SimSpec
└──► NN

Note That I did not include BRDF correction because it would generate too many options!!! In fact when will become the time for the Matchup with Satellite data, we could apply BRDF correction using the THOMAS python package….

Automated Configuration Updates & Traceability:
When you select one of these 9 options via the --version flag, you don't need to manually edit any files. The main script automatically opens the master configuration file (pySAS_Amundsen_2026.cfg) and the SeaBASS metadata file (pySAS_Amundsen_2026.hdr), translates your choice into the corresponding binary switches (0 or 1), and overwrites the master files inside the /Config/ directory so HyperCP executes the correct model.
To ensure reproducibility and total traceability for future analysis, the script also creates a timestamped copy of the exact configuration and header used for that specific run (e.g., config_run_M99SimSpec.cfg and header_run_M99SimSpec.hdr) and saves them directly inside the corresponding Level-2 output directory alongside your final processed .hdf files. This allows you to verify months later exactly what physical assumptions were made for any specific dataset.

L2 Diagnostics & Quality Assessment

- **compare_L2_versions_v2.py :**
  - **Automated Multi-Method Matrix Grid**: The script automatically maps and processes all available time-binned ensembles (5-minute casts) across all selected Level-2 versions simultaneously. It automatically adapts to missing or rejected L2 files (such as runs failing due to negative reflectances from over-corrected NIR offsets) without crashing the batch execution.
  - **Dynamic QWIP-Driven Legend Sorting**: For each independent time slice, the routine extracts the Quality Water Index Parameter (qwip) matrix from the DERIVED_PRODUCTS group. It dynamically calculates and sorts the figure's legend from the lowest (best performing model) to the highest QWIP score, instantly highlighting the optimal L2 method for any given station geometry.
- **extract_l2_qc_tables.py :**
  - **Robust Non-Parametric Rank Indexing**: To aggregate global daily performance without being skewed by local environmental spikes (e.g., surface foam or exceptional glint events), the extraction tool scores each processing variant with a relative rank from 1 to N for every single cast.
  - **Leaderboard Generation and GIS Export**: It compiles these scores into two standalone tabular files: a global cross-pivoted matrix (L2_QC_Global_Summary_[DATE].csv) and an automated leaderboard (L2_Global_Leaderboard_[DATE].csv) computing the daily average rank quote to pinpoint the overall winning algorithm. It seamlessly pairs these tables with a spatial tracking layer (.geojson) containing the ship's corrected ANCILLARY coordinates for instant mapping in standard GIS software like QGIS. See the example below.


PostgreSQL
STAC
MS SQL Server
Oracle
WMS/WMTS
Cloud
Scènes
SensorThings
Tuiles vectorielles
Tuiles XYZ
Mapzen Global Terrain
OpenStreetMap
WCS
WFS / OGC API - Features
Serveurs ArcGIS REST
Couches
pySAS_L2_Track_QC_20260701
3CNIR
3CSimSpec
M99NN
M99SimSpec
Z17NN
Z17SimSpec
OpenStreetMap
Estuaire moyen du Fleuve Saint-Laurent
Québec
Beauport
Montmagny
La Malbaie
La Pocatière
Saint-Pascal
ZEC Chapais
Île d'Orléans